

Exam questions – Computer Networks

Juni 2024

Application layer

1. Peer-to-Peer Chat Application (10)

Propose a high-level architecture for a Peer-to-Peer (P2P) chat application that runs on top of one or more of the P2P networks that we studied (Napster, Gnutella 0.4, Gnutella 0.6 and BitTorrent).

A. Describe the phases of operation for your P2P chat application.

B. Which P2P network is used for each phase of operation.

Make sure to justify your selections.

When a user wants to join the network, we use the **Gnutella discovery protocol** to find neighbors.

- The user sends a PING message to a well-known node, which forwards this to all its neighbors using flooding.
- The neighbors answer with PONG and additional information such as their IP address and the port they're listening on. The new user picks 4 of them to keep for redundancy.
- A **Napster**-style approach could also work, with central index servers that contain information of each user (username and password). **Gnutella**, however, provides a fully **decentralized** discovery phase.

The user can send messages to any user in the network. Gnutella's protocol is also suited for this task.

- Users flood a QUERY into the network. All nodes are required to forward this QUERY. If the correct node (user) is found, it responds with a QUERYHIT with the address of the user.
- A **TTL (Time to Live)** counter is used together with a **unique identifier** to limit flood overhead and loops.
- The user can now open an **HTTP connection** with the user and exchange messages. HTTP is simple but effective enough: messages are just text and do not contain large amounts of data.
- If the HTTP connection fails, a PUSH message is sent to the receiver so it can disable its firewall for that port.
- **Gnutella 0.6** can be used to optimize this discovery process. It differentiates between **super users** (lots of RAM, bandwidth and no firewall) which can take on the tasks of discovery and **leaf nodes**, which only tell the super users which user they are. This is essential for our chat application as we expect a lot of traffic between users.
- **Group chats** can be supported using multicast.

Chat history can be saved in a **BitTorrent**-style network. Users can encrypt the history if they own the **chat key**.

- The chat key is generated using **asymmetric encryption**. When starting a new chat, the sender generates a chat key and sends it to the receiver, encrypted using the receiver's public key. This chat key can then be used to encrypt and decrypt the chat history.
- Same holds for group chats!
- BitTorrent ensures the efficient distribution of chat histories
- (Also explain how BitTorrent works)

2. Attacking the TOR network (10)

You are tasked to surveille the TOR network. Provide a high-level overview of how you would spy on the network in the case that you are provided with:

A. Limited resources (for example in a research project).

B. Unlimited resources (for example as a powerful state actor).

The anonymity in the **TOR** network is ensured by creating a route with at least 3 router nodes. Each step is then encrypted using that nodes' public key (each hop negotiates a fresh key and TLS session to further improve safety). This means that each node can only 'peel off' one layer of the 'onion' using their private key. The last node then forwards the content to the receiver. This means:

- **Entry nodes** only know who sent the file
- **Middle nodes** only know the previous and next hop
- **Exit nodes** only know what the contents of the file are and who to send it to, but not where it came from.

We will surveille the TOR network using a **Sybil-style attack**. By creating 'fake' nodes we, hopefully, can infer information about the content and users that are using the TOR network. (It is also possible to generate fake information etc., but this is not needed in this experiment.)

With limited resources, we can infer **what** is sent over the network by creating as many **exit nodes** as we can. This allows us to monitor the content that is sent over the TOR network, although it might be encrypted.

We can link traffic to senders by also creating as many **entry nodes** as possible. If we are lucky, a route consists of one of our entry nodes and exit nodes. Since TOR connections are TCP/IP connections under the hood, we can assume they are a bit 'predictable' (with heavy emphasis on 'a bit').

- TCP/IP connections are session based and **connection-oriented** which means they will follow the same route through the network (if we ignore congestion and a lot of other small nuances).
- By tracking timestamps and packet sizes, we can possibly correlate an entry node and exit node. Then, we can link the data with its sender, though this is statistical.
- We can improve our 'consensus' by provided high-capacity links.

With unlimited resources, we could try to take over the whole network (or at least a large part) and add a lot of **high-capacity relays**. If we are lucky, a whole onion route goes through our nodes, which means we can fully infer and link all information about data and senders. Nations can target critical infrastructures such as IXPs for tier-1 carriers across multiple ASes to increase the chance traffic needs to flow through one of our nodes.

Transport layer

1. QUIC (10)

Provide a high-level overview of the design of QUIC and highlight the major differences between QUIC and TCP.

The internet needed improved infrastructure for modern problems:

- Devices are more **mobile** (e.g. laptops, phones). These also offer multiple **network interfaces** (e.g. 4G, WiFi)
- Applications are becoming **multimedia** (e.g. websites having images, videos etc.)
- TCP offers extensibility through options, but these are under-utilized, not required and, therefore, often just discarded.

The main design goals of QUIC are:

- **Built-in encryption** in transport protocol
- Support for **multimedia** using **streams**. This means different traffic can flow in different rates. In TCP, all traffic needed to be sent sequentially, making it difficult to build parallelized structures.
- **Low-latency connection establishment** (no more 3-way handshake)
- **Connection migration**
- Implemented in user-space on top of UDP to encourage fast adoption.

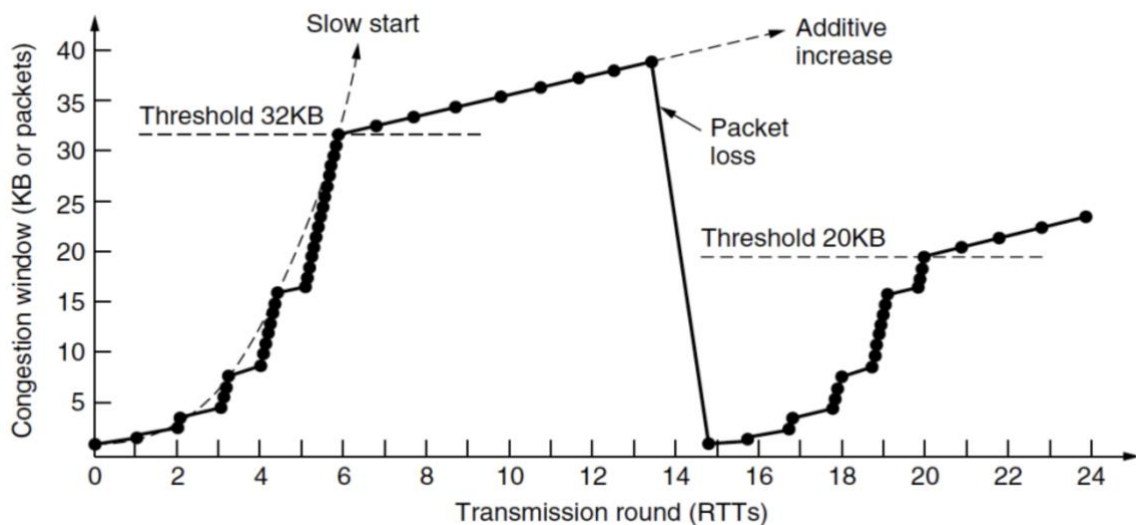
TCP	QUIC
32-bit sequence number for each byte of data, so wraps around every 4GB.	62-bit sequence number, meaning it never wraps around, and each byte is uniquely identified.
Sliding window protocol: selective repeat, meaning we have both accumulative ACKs and negative ACKs. TCP Reno also supports selective ACKs, although limited by option header size.	Selective ACKs are supported and unlimited.
No multiplexing of multimedia streams, so slower streams congest the whole data.	Multiplexing built in. User can create and manage multiple types of streams.
3-way handshake needed (although reduced in newer versions of TLS)	1-way handshake or even 0-way handshake if both hosts already established an authenticated connection before ('Early data')
Congestion control using $RTT = SRTT + 4 \cdot RTT_{VAR}$	Congestion control using Cubic.

No connection migration possible

Connection migration allows a change of IP and port while maintaining the connection. Done by temporarily reducing the send rate until the new connection is verified.

2. TCP Tahoe Congestion Control (6)

Provide a graph showing the operation of the Transmission Control Protocol (TCP), Tahoe congestion control algorithm. Annotate each phase of the graph to show what is happening.



- At the start of a connection, we perform **slow start** (terrible name). We exponentially increase the number of packets sent (1, 2, 4, 8) until congestion is detected.
- We need a **threshold** to keep slow start in check, because congestion will occur quickly and eventually collapse. The initial threshold is set to the **flow control window size**. This means only a slower network in-between will cause congestions, and not because of the small receiver.
- When the threshold is reached, TCP switches to **AIMD** (additive increase, multiple decrease).
- If packet loss occurs, we restart the slow start.
- We can optimize this! Instead of starting from scratch, we can just wait until the number of packets falls below a new threshold (about $\frac{1}{2}$ RTT). Then we can start additive increase again.

3. Congestion Control: ECN (4)

Describe the operation of Explicit Congestion Notification (ECN) in as much detail as you are able. Provide one advantage and one disadvantage of ECN in comparison to RED.

- **ECN = Explicit Congestion Notification.** If a router (network layer) approaches congestion, it will flip a bit in the TCP header to notify the upper layers to slow down sending. A receiver that gets such a message will then notify the sender to slow down.
- **RED = Random Early Detection.** Instead of an explicit notification, the router just randomly drops packets in the hope that upper layer protocols notice the increase in packet loss and RTTs.

-	+
The delay between the notification and the sender slowing down is relatively large, because the receiver first needs to get the ECN and then notify the sender. ECN only works if both sender and receiver support it (negotiated at start). RED is way quicker and makes packet loss a viable congestion signal. Also punishes fast senders because they will have many packets in the queue.	ECN avoids costly retransmissions, because the packets aren't lost, just marked.

Network layer

1. MTU Discovery and IPv4 Fragmentation (10)

In its current form, the processes of Maximum Transmission Unit (MTU) discovery and IPv4 Fragmentation are mutually incompatible.

- Why are these two processes incompatible?
- Suggest modifications to make these processes compatible.
- How efficient is your proposed approach?

MTU discovery will send packets of a certain length into the network to see if they get rejected (ICMP responds with 'fragmentation needed'). They will try this until they find the MTU. This requires routers to deny packets that are too large and optionally send an error message containing the maximum packet size. **Fragmentation** will prevent this from being possible, since instead of rejecting, they will fragment the packet and rebuilt it at the next router/host (**transparent** vs. **non-transparent fragmentation**). A 'DF' = 'Don't Fragment' flag notifies the router not to fragment.

Proposed solutions:

- When fragmentation occurs, the router prepends the max. fragment size to the data. This flag stays there until the packet is received by the host. If a router's max. fragment size is smaller, it changes this number. This could be part of the header, since it's a fixed-size number.
- If the current packet is a MTU discovery packet (for example a separate flag), the first fragment is sent separate from the rest. A later router might need to fragment again, which means the total size of the packet will be reduced to the MTU that is

possible in this network. The host will then receive a packet with the length of the MTU. The other packets will still be defragmented like normal and arrive later.

- To remove the need for a flag, you could do this always instead of requiring a flag to be set. This way, they work together seamlessly.
- Fragmentation always happens on top of the ICMP message being sent. So the sender knows their packets are too large, but the packet still arrived correctly.

The first approach is the easiest but adds a fixed-size number to the header for MTU discovery, which is only a marginally small fraction of the traffic.

The second approach allows for fragmentation and MTU discovery to work together but does mean there is more ‘unnecessary’ overhead in the network, because the first fragment is sent as a fully separate packet from the rest, meaning it will not get merged at the end, and just stay a small packet.

The last proposal requires the least changes but does generate a duplicate packet in the network each time an upper layer protocol does not abide to the MTU.

2. The Internet Protocol, Version 6 (IPv6) (10)

Describe the privacy problems that arise from IPv6 address auto-configuration.

- A. Why is more information revealed when using auto-configuration to generate your IPv6 address than other mechanisms such as DHCPv6?**
- B. Propose an experiment to measure the proportion of hosts that are using IPv6 auto-configuration. Your experiment can assume access to the traffic passing through core Internet routers, but not the hosts themselves.**
- C. Comment on the expected accuracy of your experiment.**

To generate an IP address, often the MAC address is used in **auto-configuration**. This is unsafe, as MAC addresses are physical addresses, meaning they can be static and topologically linked by correlating traffic. **DHCPv6** is much safer as it will lease **temporary** IP addresses to the user that expire after a certain amount of time.

If all hosts’ IP addresses are known, we can perform the following actions:

- All core routers have AODV, which means they will discover routes reactively and purge routes if they are not found anymore.
- Continuously send packets to each IP address. If the IP address is auto-configured and based on a static MAC address, chances are that it doesn’t move. This means that the route will eventually be discovered by AODV and stored in the core routers.
- IP addresses generated by DHCP however will expire and therefore be unreachable from time to time. This means the route needs to be purged from the AODV router.
- Addresses that are purged more often have a higher chance to be auto-configured.

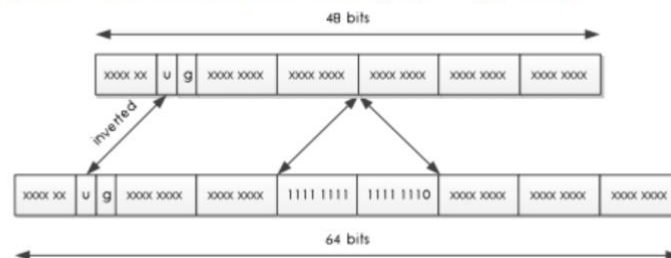
The efficacy of this method is limited:

- Reachability does not necessarily correlate to configuration time: DHCPv6 may automatically renew leases for extended periods of time, and SLAAC addresses may go offline (e.g. phone changes network)
- Core routers do not typically run AODV. It was not really implied that it was possible to assume this...

A better solution:

- MAC-generated IPv6 addresses follow a specific conversion scheme. The MAC address can be inferred from this.

Create a IID (EUI-64 modified) from a MAC address (48 bits):



- Monitor IPv6 source addresses on core links and extract their 64-bit interface IDs (lower 64 bits)
- Classify any IID that embeds 'FFFE' in the middle as SLAAC.
- This only requires the header and no additional traffic. The accuracy depends on the chance that a DHCPv6-generated address also has 'FFFE' in the middle.

Data link and physical layer

1. Which Wireless Medium Access Control (MAC) protocols are a good fit with Ad-Hoc On-Demand Vector (AODV) routing? (6)

We studied various MAC protocols for wireless networks. Some of these would be expected to work well with AODV, while others would not.

A. List two MAC protocols that would work well with AODV.

B. List two MAC protocols which are a bad fit with AODV.

AODV routing will discover (smallest) routes to nodes in the network **reactively**, which means only when a packet is sent in the network. If the cache does not contain the address:

- It will flood the network with a ROUTE_REQUEST message (accompanied by an increasing TTL and sequence number).
- Discovered nodes send a ROUTE_REPLY and increase the hop counter.
- Intermediate nodes also add the route to their cache.

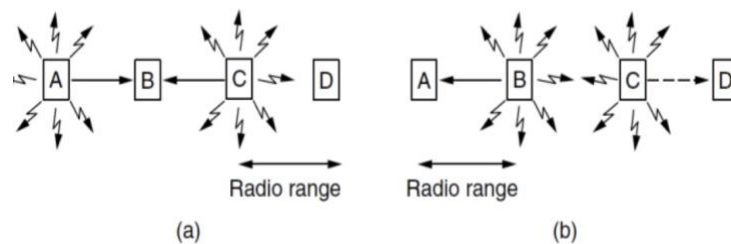
Work well with AODV	Don't work well with AODV
1-persistent or P-persistent CSMA: carrier sense reduces collisions and samples continuously.	TSMP: joining the network is high cost because it requires you to synchronize with the other connections. Goes against the dynamic, reactive nature of AODV.

BMAC: the LPL complements AODV's reactive nature perfectly and reduces collision.

Pure Aloha: extremely high collision rate in dense areas. There will be a lot of 'advertising' by the routers, so a lot of traffic will be generated just to route.

2. The Hidden and Exposed Terminal Problem (4)

Illustrate the hidden and exposed terminal problems using a diagram and supporting text.



Even if we implement **carrier sense** to avoid collisions, we still run into problems:

- (1) **Hidden terminal:** terminals A and C are out of range and think there will be no collisions, so they both try to send data to terminal B, which will experience contention.
- (2) **Exposed terminal:** terminal B and C are in each other's range but want to send data to resp. A and D, who are out of each other's range. They think collision will occur and will wait for no reason.

3. Ethernet Switches and Hubs (6)

Describe how an Ethernet switch and hub work.

A. What are the advantages of a switch over a hub?

B. What type of experimentation might a hub support that a switch cannot?

Hub:

- 'Merges' multiple lines together
- Sends all received frames to all lines connected on the hub.
- Does not prevent collisions from occurring.
- Only prevents the problem of 'single point of failure'.

Switches

- High-speed backplane to connect hosts on a single line. Each cable is fully duplex
- Store a table with MAC-addresses. Only the MAC-address the frame belongs to will receive the frame.

Switches have multiple advantages over hubs:

- If the lines are not connected to hubs and are full duplex, CSMA/CD is not necessary as all lines will be in their own collision domain. This improves speed.
- Improves capacity as frames are only delivered to intended hosts.

- Impact of promiscuous mode is limited.

Hubs can be used to debug the networks and perform tests, e.g. testing how collision signals manifest in the physical layer, test different carrier sense thresholds, test the impact of flooding and the throughput of the line, etc.

Also, for LANs, one could argue that promiscuous mode does not impose a problem, but this doesn't necessarily mean it is better than switches.

4. The ALOHA Protocol (4)

Would it be possible to re-implement the Pure ALOHA MAC protocol on the unmodified 802.11 physical layer? Justify your answer.

The **802.11 physical layer** provides the fundamental capabilities for wireless transmission and reception. It can modulate data onto radio waves, transmit them, and receive them. It also has basic "**Clear Channel Assessment**" (CCA) functionality, but for **Pure ALOHA**, you'd essentially instruct the PHY to ignore the CCA outcome.

While it is technically possible, you would essentially be 'wasting' the physical capabilities (CCA) and the numerous data-link layer capabilities (NAV, CSMA/CA, RTS/CTS, ACKs, etc.), which is not useful at all.